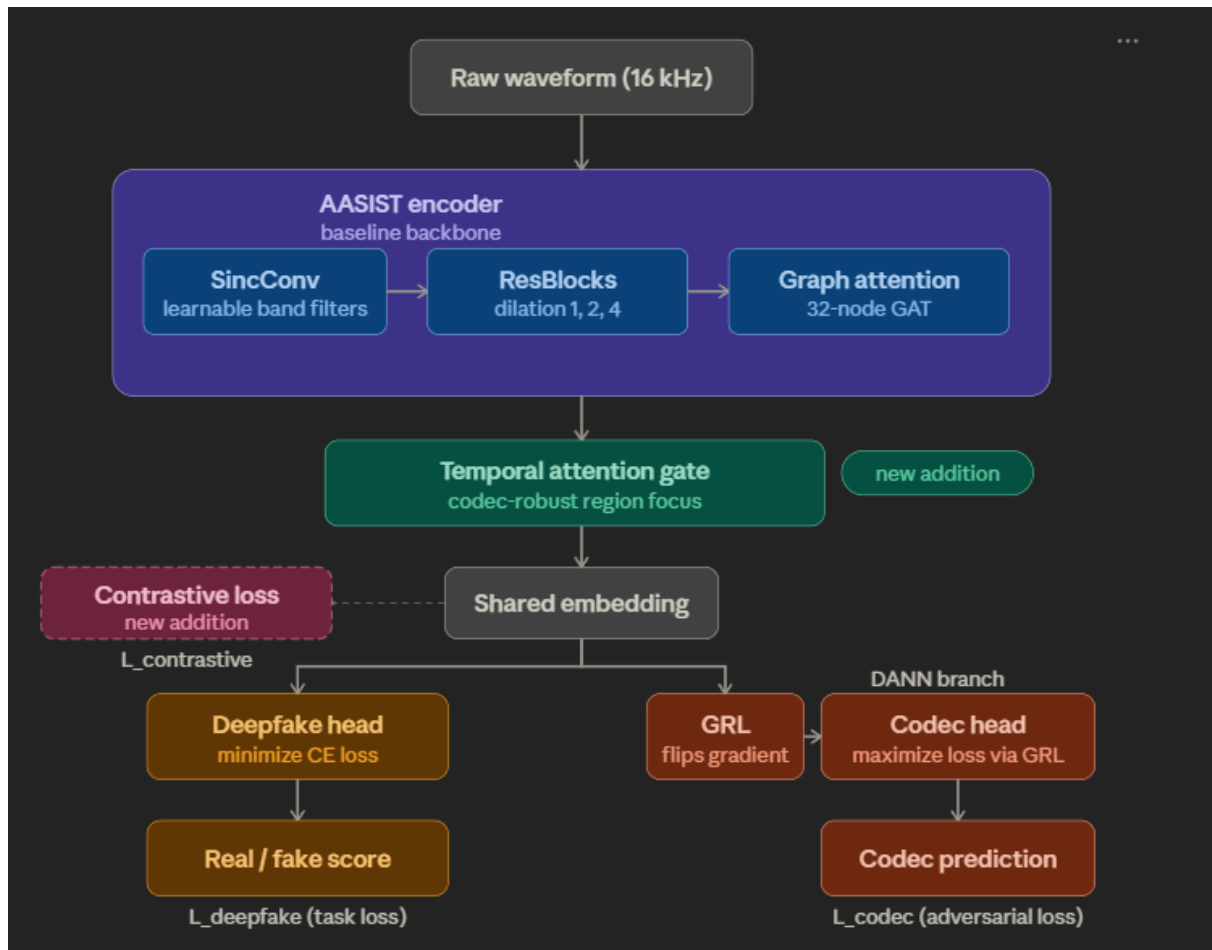


DL project-2:

Deepfake detection improving on AASIST:

Design:



The baseline: AASIST

AASIST is a state-of-the-art deepfake audio detector from 2022. It takes a **raw waveform** as input — no hand-crafted features. It has three stages inside:

SincConv — Instead of a regular conv layer, this learns bandpass filters directly on the waveform. Each filter has a learnable low cutoff and high cutoff frequency. Codec compression destroys specific frequency bands — this layer learns which bands matter for detecting fakes.

ResBlocks — Stacked dilated 1D conv residual blocks (dilation 1, 2, 4 gives exponentially growing receptive field). They capture temporal patterns at multiple scales without losing resolution.

Graph Attention (GAT) — This is what makes AASIST distinctive. The 1D sequence is pooled into 32 "nodes" and a graph attention network models *relationships between temporal regions* — not just individual frames. Real speech has specific co-dependency patterns that fakes violate.

The output is a **256-dim embedding vector** per clip.

The problem AASIST alone has

AASIST was trained on clean, uncompressed audio. When you run MP3 or Opus-compressed audio through it, its embedding shifts — it picks up the codec fingerprint as a confounding signal. A real voice through Opus-32kbps lands in a different region of embedding space than the same voice uncompressed. The detector gets confused.

① Temporal Attention Gate

After AASIST produces the embedding, we feed it through a **transformer attention block** before the deepfake head. This lets the model learn to attend to the parts of the representation that are *stable across codecs* — the fundamental pitch, prosody patterns — and suppress the high-frequency spectral artifacts that codecs introduce. Think of it as a learned filter that says "ignore the codec noise, focus on the voice".

DANN: Gradient Reversal Layer + Codec Head

This is the core architectural trick. We add a **second classifier** that tries to predict *which codec and bitrate was used*. In normal training this would just learn codec features — but we insert a **Gradient Reversal Layer (GRL)** between the embedding and this classifier.

Here's the key insight: during the **forward pass** the GRL does nothing — it just passes data through. But during **backward pass** when gradients flow back, it *multiplies them by $-\lambda$* — flipping the sign. So instead of the encoder learning to be *better* at identifying codecs, it learns to be *worse* at it. The encoder is being pulled in two directions at once:

- The deepfake head says: "learn features that separate real from fake"
- The codec head + GRL says: "unlearn any features that reveal the codec"

The result: the embedding becomes **codec-invariant**. The robustness is baked into the representation, not patched on.

The λ (lambda) value starts near 0 and gradually increases to 1.0 — if you crank it up too fast, training destabilizes.

② Contrastive Loss

This works alongside the GRL. We take the *same utterance*, compress it with two *different* codecs, run both through the model, and train the embeddings to be **close together** in embedding space. This directly enforces what we want: same voice = same embedding, regardless of codec. The GRL says "forget codec info", the contrastive loss says "here are examples of what should look the same".



Total training objective

$$L_{\text{total}} = L_{\text{deepfake}} + \alpha \times L_{\text{codec}} + \beta \times L_{\text{contrastive}}$$

- **L_{deepfake}** — cross entropy on real/fake (standard classification)
- **L_{codec}** — cross entropy on codec type, but the GRL makes the encoder try to *maximize* this (adversarial)
- **$L_{\text{contrastive}}$** — InfoNCE loss pulling same-utterance-different-codec pairs together

$\alpha = 0.5$, $\beta = 0.1$ are the default weights. You'll tune these.

What your experiment shows

You train two models and compare:

1. **Baseline** — AASIST alone, no codec augmentation, no DANN
2. **Ours** — AASIST + temporal attention + GRL + contrastive, trained with random codec augmentation

Short explanation of the components:

Here are the punchy, one-line mental hooks to help these concepts stick in your head:

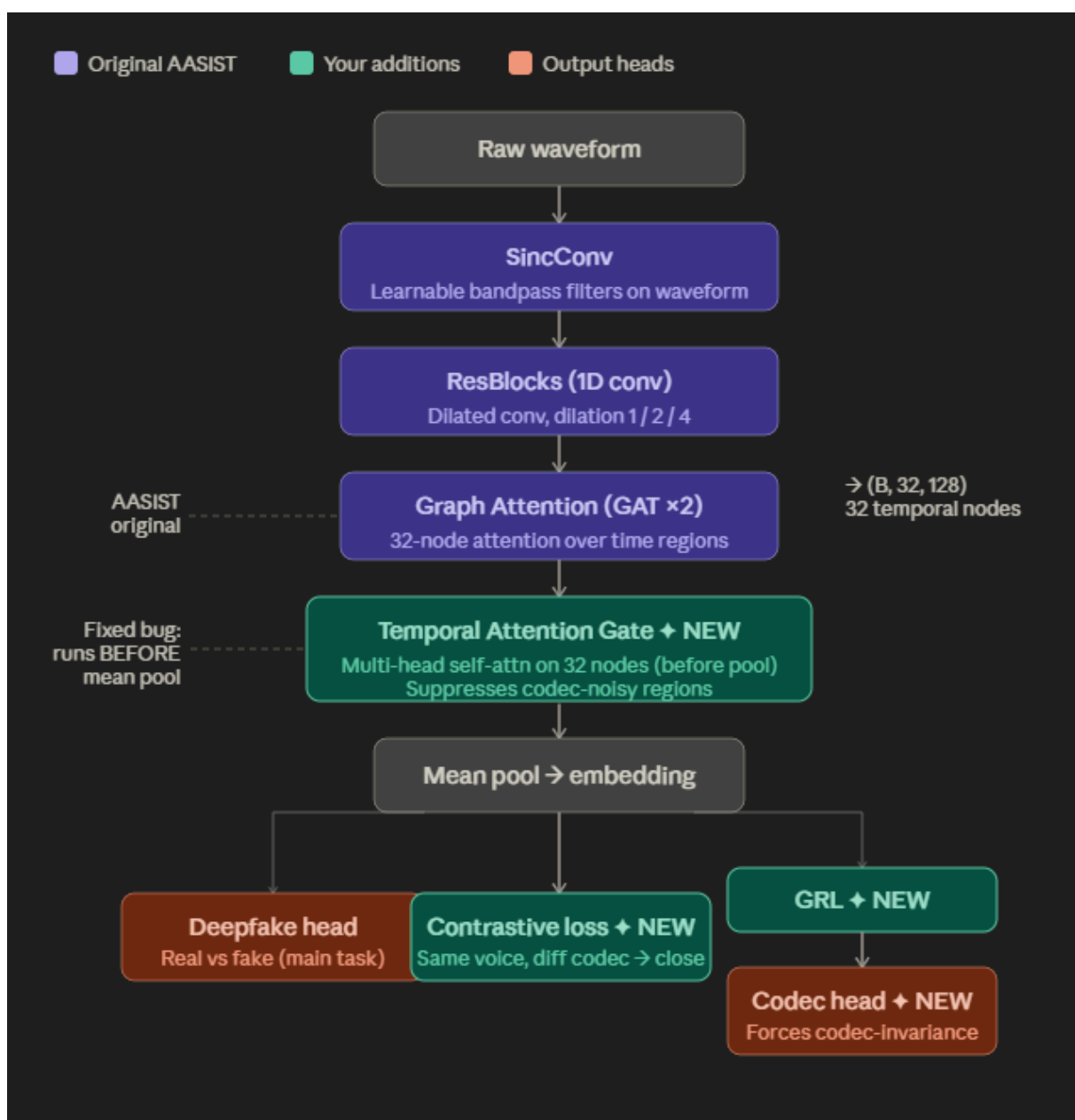
- **AASIST (The Baseline)**: The core engine that reads raw soundwaves and turns them into a unique 256-number digital fingerprint.
- **SincConv**: A smart equalizer that automatically tunes itself to hunt down the specific audio frequencies where deepfake glitches hide.
- **ResBlocks**: A multi-scale scanner that checks for robotic patterns in both split-second syllables and the broader rhythm of a whole sentence.
- **Graph Attention (GAT)**: A rhythm detective that analyzes how different chunks of the audio relate across time to catch unnaturally stitched speech.
- **Temporal Attention Gate**: A learned spotlight that explicitly dims the useless static of file compression so the model can focus purely on the voice.
- **Gradient Reversal Layer (GRL)**: A trick mirror that actively punishes the model whenever it tries to memorize file formats, forcing it to become "codec-blind."
- **Contrastive Loss**: A rubber band that physically forces a compressed MP3 and a clean WAV of the exact same voice to look perfectly identical to the model.

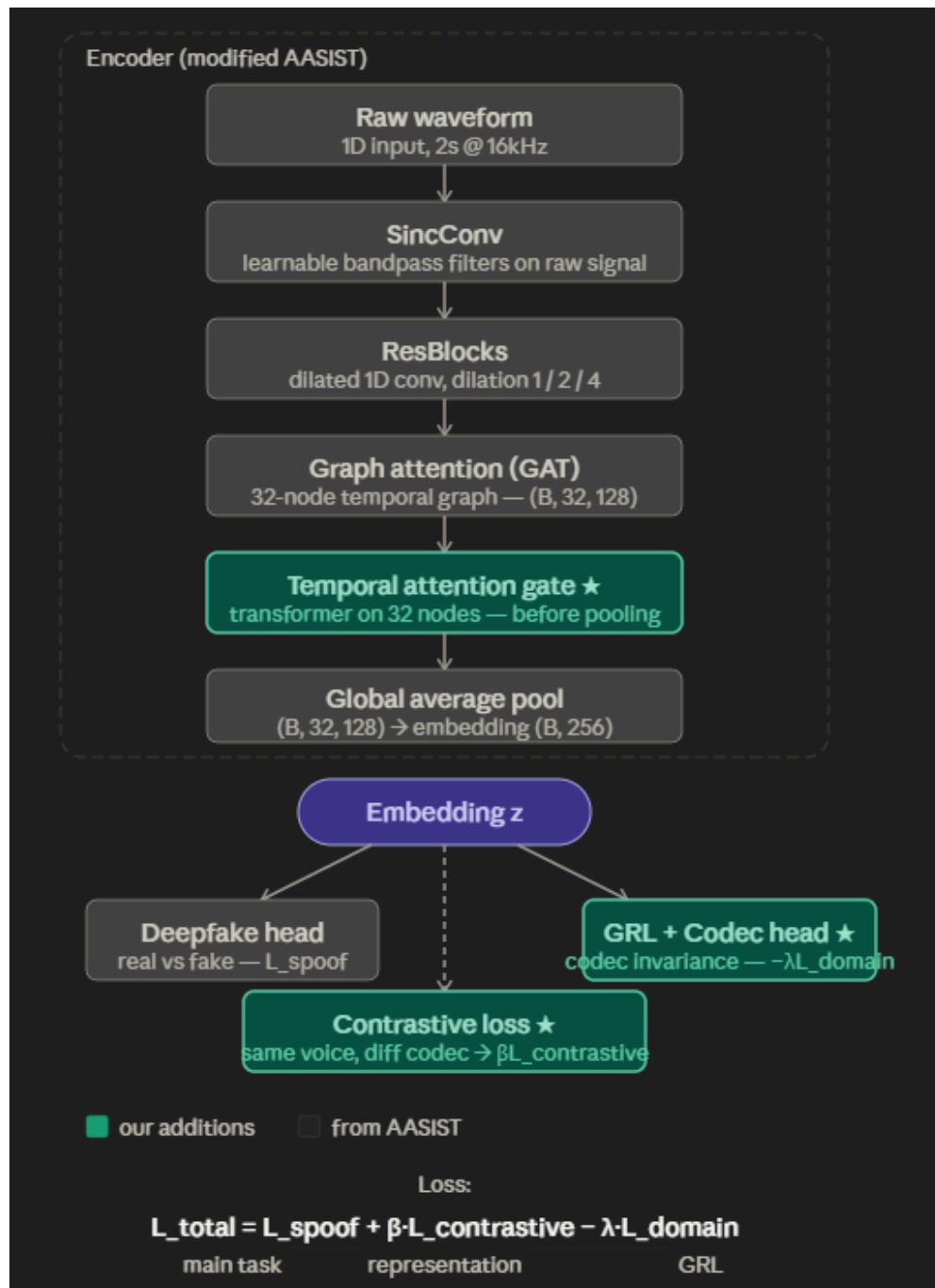
real world metrics:

What are your success metrics?

Our goal is to close the gap between clean-audio and compressed-audio detection performance and possibly try to outperform AASIST

20-4 : Monday:





temporal:

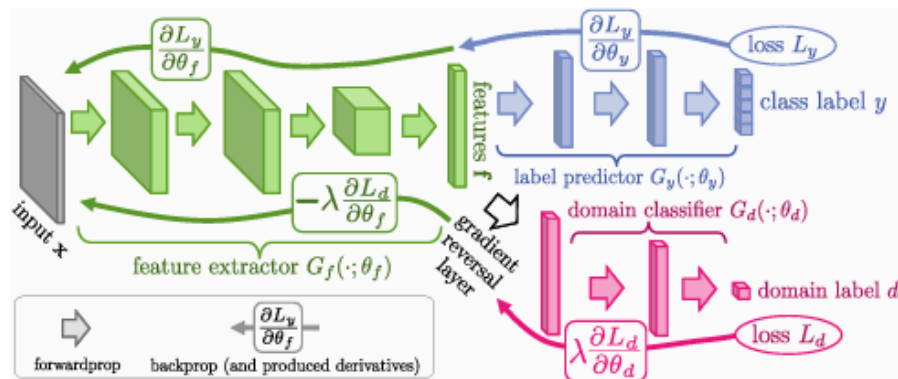
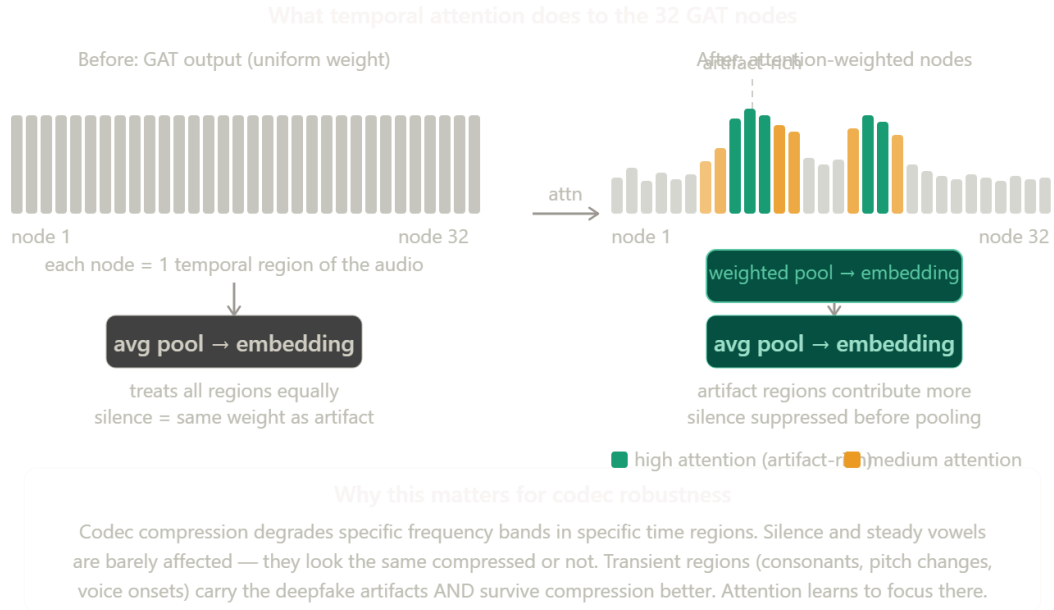


Figure 1: The **proposed architecture** includes a deep *feature extractor* (green) and a deep *label predictor* (blue), which together form a standard feed-forward architecture. Unsupervised domain adaptation is achieved by adding a *domain classifier* (red) connected to the feature extractor via a *gradient reversal layer* that multiplies the gradient by a certain negative constant during the backpropagation-based training. Otherwise, the training proceeds standardly and minimizes the label prediction loss (for source examples) and the domain classification loss (for all samples). Gradient reversal ensures that the feature distributions over the two domains are made similar (as indistinguishable as possible for the domain classifier), thus resulting in the domain-invariant features.

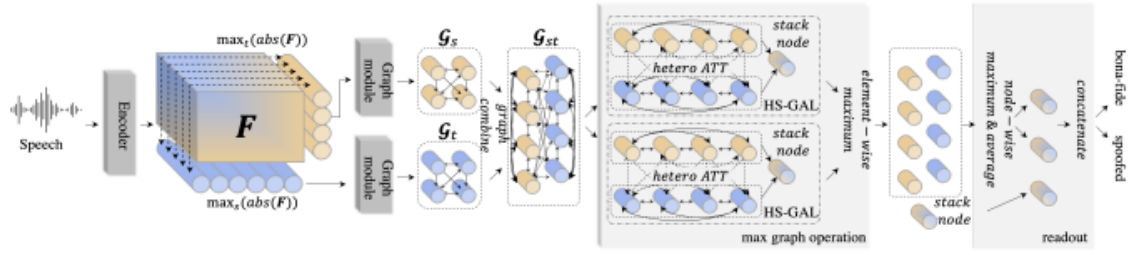


Fig. 1. Overall framework of the proposed AASIST. *Identical to [11]*: encoder extracts F and two graph modules each model spectral and temporal domains. *Proposed*: then, the proposed max graph operation technique adopts two branches that model heterogeneous graphs in parallel, followed by an element-wise maximum. Each branch includes two proposed HS-GAL layers and two graph pooling layers (graph pooling layers and one HS-GAL layer is omitted in the illustration). Finally, the maximum and average of nodes, and the stack node are concatenated followed by an output layer.